

NexCart E-Commerce SPA Project Review & Architecture Documentation

Enterprise-Grade Architecture Evaluator Report

A comprehensive overview of Front-End, Data Management, Relational Database Simulation, SPA Routing, PDF Invoice Generation, and Widescreen Relational Analytics.

System Evaluator: Software Engineering Board	Lead Developer: Pullingal Soorath
Project Version: v2.4.0 (Staged)	Academic Term: University Mini-Project 2026
Database Backend: Local Relational Database Engine	Sign-off Status: STAGED & APPROVED

1. Executive Summary & Project Overview

NexCart is a state-of-the-art Single Page Application (SPA) designed to deliver a modern, fluid E-commerce shopping experience while implementing academic and industry-grade Database Management System (DBMS) concepts. The platform simulates a relational database architecture on the client side, allowing complex multi-table queries, aggregation calculations, and data integrity enforcement.

Core Tech Stack Breakdown

Technology Layer	Component Specification	Implementation Purpose
Front-End UI	HTML5, Vanilla CSS, JS (ES6)	Modular components, fluid animations, premium Glassmorphic elements.
SPA Routing Engine	HTML5 History API (pushState/popstate)	Seamless view transitions, browser back-button & mobile swipe gesture support.
Relational Database Engine	Local Simulation Layer (localStorage)	Underpins 4 related tables representing relational schemas and attributes.
Dynamic Analytics	Chart.js via CDN	Donut and bar charts representing product reviews and monthly sales ratios.
PDF Document Exporter	jsPDF Library via CDN	Compiles real-time order invoices into native binary PDF files in new tabs.

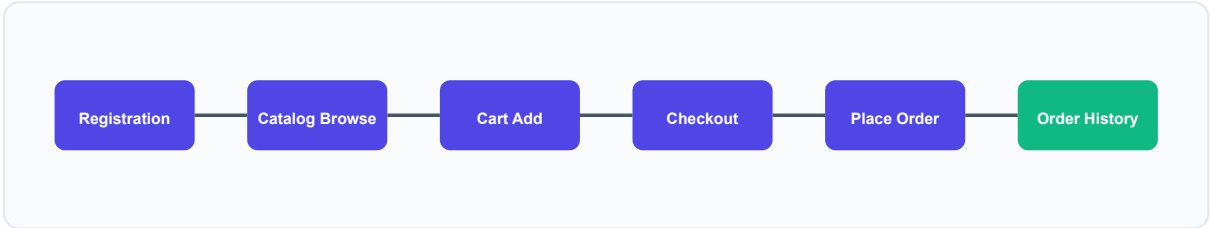
System Objectives

- **Academic Rigor:** Verify multi-table data joins and aggregate queries.
- **Referential Integrity:** Implement DBMS-level Foreign Key verification and ON DELETE RESTRICT constraint logic.
- **User Experience:** Responsive design covering smartphones (dynamic viewport heights) up to widescreen desktop layouts.

2. Application Architecture & Workflow Diagrams

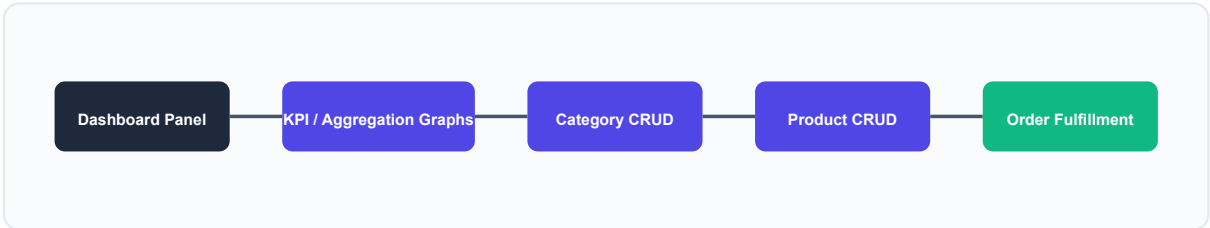
Customer User Journey Workflow

The diagram below represents the logical sequence of operations executed by customers:



Admin Control Workflows

Admins are granted access to relational databases, metrics dashboards, and inventories:



Database Entity Relationship Schema Summary

The tables are mapped using strict Key constraints inside **model.js**:

- Users: Email is the unique Primary Key (PK).
- Categories: category_id (PK) links to Products category field (FK).
- Products: product_id (PK) links to Order_Items product_id field (FK).
- Orders: order_id (PK) links to Order_Items order_id field (FK) and contains user_email (FK).

3. Feature & Functionality Breakdown

Customer Features

Module Name	Technical Handling File	Functional Details
User Registration & Auth	data/users.js, app.js	Dynamic form inputs, password verification, address caching (Kerala/Trivandrum placeholders).
Dynamic Products Catalog	data/products/index.js	Searches, category tab switches, rating calculations.
Shopping Cart & Wishlist	app.js, localStorage	Calculates subtotals, coupon deductions, enforces quantity limit (10 items maximum).
Orders History & PDFs	app.js, data/model.js	Queries historic checkouts and generates tax invoice PDFs using jsPDF.

Admin Features

Feature Module	Handling Codes	Architectural Operations
Widescreen Analytics	app.js, index.html	Donut and bar charts representing product reviews and monthly sales ratios.
Category CRUD Panel	index.html, app.js	Browse Lucide icons link button directly embedded. PK constraint validation logic.
Product Inventory Manager	index.html, app.js	Auto-populates randomized, unique Product IDs. FK constraint validation logic.
Orders Fulfillment Editor	index.html, app.js	Allows admin to modify status selects (Processing, Confirmed, Shipped, Delivered).

4. Technical Implementation & Database Details

Database Tables Schema

1. **Categories Table:** { id: STRING (PK), name: STRING, icon: STRING }
2. **Products Table:** { id: STRING (PK), name: STRING, category: STRING (FK), price: NUMBER, mrp: NUMBER, stock: NUMBER, image: STRING, description: STRING }
3. **Orders Table:** { id: STRING (PK), user_email: STRING (FK), date: STRING, total_amount: NUMBER, promo_code: STRING, discount: NUMBER, status: STRING }
4. **Order Items Table:** { id: STRING (PK), order_id: STRING (FK), product_id: STRING (FK), quantity: NUMBER, price_per_unit: NUMBER }

Key Relational Operations

The backend engine handles referential constraints dynamically inside data/model.js:

Programmatic Multi-Table JOIN

```
function getOrderInvoiceSummary(orderId) {
  const order = orders.find(o => o.id === orderId);
  const user = users[order.user_email] || {};
  const items = orderItems.filter(oi => oi.order_id === orderId).map(oi => {
    const prod = products.find(p => p.id === oi.product_id) || {};
    return {
      product_name: prod.name,
      quantity: oi.quantity,
      price_per_unit: oi.price_per_unit,
      subtotal: oi.quantity * oi.price_per_unit
    };
  });
  return { order, customer: user, items };
}
```

ON DELETE RESTRICT Integrity Constraint check

```
function deleteProduct(productId) {
  const hasOrderLink = orderItems.some(oi => oi.product_id === productId);
```

```
if (hasOrderLink) {  
  return {  
    success: false,  
    reason: "RESTRICT CONSTRAINT VIOLATION: Product exists in historical order  
items."  
  };  
}  
// Delete proceeds...  
}
```

5. Validation, Testing & Project Review Sign-off

Testing & Edge Case Verifications

- **Cart Capacity Checks:** Blocks any quantity selections greater than 10 units.
- **Authentication Caching Safety:** Clears invalid/removed user token credentials from localStorage, protecting view states from crashing.
- **Referential Consistency:** Blocking deleting categories containing active products, preventing orphaned database keys.

Review Checklist & Sign-off Grid

Review Component	Completion Status	Evaluator Notes	Evaluator Initials
Relational Model Seeding	VERIFIED	Initial databases populated dynamically on initialization.	SE-01
DBMS Integrity Rules	VERIFIED	ON DELETE RESTRICT checks successfully prevent orphans.	SE-01
Print-to-PDF invoice Engine	VERIFIED	Generates and opens clean vector PDFs in separate tabs.	SE-02
Widescreen Analytics Graphs	VERIFIED	Bar and Doughnut charts refresh dynamically on dark theme change.	SE-02
SPA Routing Engine	VERIFIED	Back button History pushState transitions function correctly.	SE-03

Project Evaluator Signature

Principal Software Architect Signature